

You completed this test on *25/02/2022, 14:57*
Your score is 94.12%

Deep CNNs are susceptible to overfitting. Can you suggest some ways to overcome that?

Yes Deep CNNs are susceptible to overfit. There are many ways to overcome overfitting they are:
Regularization There are 3 mainly used regularization methods namely L! regularization, L2 regularization and Dropout regularization.
(All the actions are not performed on the output layer)
The L1 regularization reduces the weight values of less important features to zero there by shut off the particular neuron.
The L2 regularization which is weight decay forces the weight to decay towards zero there by turns off the neuron.
In Dropout regularization every unit of our neural network is given the probability p of being temporarily ignored in calculations. Hyper parameter p called dropout rate will be used to set probability.In each iteration, we randomly select the neurons that we drop according to the assigned probability.

Other Methods are
Early stopping if the testing loss gradually decreases and suddenly increases we need to use early stopping
We need to change the neural network architecture.
Need to train on more data

What are the important hyperparameters for a convolution layer?

Learning rate : which the algorithm learns
epochs: number of epochs
Batch size: size of mini batches
Activation function: the sigmoid relu or softmax
Number of hidden layers and units: choosing the appropriate
Initializing the Weights: so that algorithm do not overfit or underfit
Dropout for regularization

---

1. **You have an input RGB image of dimension 64*64*3. You apply a small CNN on this to extract some simple features.**

Conv(?*3*3*6)-> pooling->relu ->Conv(?*3*3*16)-> pooling->relu-> flatten
Conv stride: 1
(assume pooling window size =2*2, stride 2)
What would be the dimension of the output?
How many parameters would you need to train for your feature extraction network?
You now want to use these extracted features to build an image classifier for CIFAR-10 dataset.
What would be your next steps?
How many more parameters did it introduce?

16*16*16 would be the size of the output
we need to train 54 parameters

Max Pooling induces …… invariance(s) to the model:

Scale

Translation

Max pooling achieves partial invariance to small translations because the max of a region depends only on the single largest element.

Rotation

None of the above

Which of the following activation function(s) has no gradient for negative responses

Sigmoid

TanH

ReLU

ReLU = max(0,z) as it does not allow negative values

Absolute

In training a neural network, you notice that the loss does not decrease in the few starting epochs. The reason could be:

The learning is rate is low

Regularization parameter is high

For any of the reasons mentioned the loss might not decrease

Stuck at local minima

Any of above

Which of the following statements is true when you use 1×1 convolutions in a CNN?

It can help in dimensionality reduction

It can be used for feature pooling

It suffers less overfitting due to small kernel size

As the convolution is 1X1 it will reduce dimensionality, it is used for feature pooling and as it reduces the dimensions the possibility of overfitting reduces

All of the above

The input image has been converted into a matrix of size 28 X 28 and a kernel/filter of size 7 X 7 with a stride of 1. What will be the size of the convoluted matrix?

22 X 22

((n + p - f)/s)+1 is the formula so

21 X 21

padding p = 0 and stride s = 1 nh = 28 nw = 28 fh = 7 fw = 7

28 X 28

28-7 + 1 = 22

7 X 7

---

1. **How would you choose a kernel size for you convolution layer? What tradeoff would you consider to find an optimum one?**
   What kernel size(s) do you observe in standard CNN networks?
   Can you and would you choose an even number filter size?why/ why not?
   What does a 1*1 filter signify?

The first convolution layer is often kept larger. Its size is less important as there is only one first layer, and it has fewer input channels:
In case of small kernel sizes, one does not have to worry worry about sampling. So the overall input size is much efficient when the kernel size is small and hence it takes less tome to process and there is less ambiguity.
A common choice is to keep the kernel size at 3x3 or 5x5.
Kernel size can be even but it may end up in a error so we would not prefer even sized kernel.
The 1X1 filter will convolve over the entire input image pixel by pixel. that is it would go on multiplying with all the elements of the input image.So it would be helpful in dimensionality reduction.

---

1. **What would be the sequence of performing the following tasks in tensorflow while writing a training script for the CNN model you've made: (assume that you are running for just 1 epoch)**
   1. **Define variable & placeholders**
   2. **Initialize variables**
   3. **Feed data to placeholders**
   4. **Start the session**
   5. **Close the session**
   6. **Run optimizer**

7. **Generate a summary of your important metrics**
8. **Calculate loss**

Define variable and placeholders
Initialize variables
Feed data to placeholders
start the session
run optimizer
generate a summary of your important metrics
calculate loss
close the session

---

1. **Given below is an input matrix of shape 7 X 7. What will be the output on applying a max pooling of size 3 X 3 with a stride of 2?**

max(this 3X3 matrix is 4)

1	2	4	1	4	0	1
0	0	1	6	1	5	5
1	4	4	5	1	4	1
4	1	5	1	6	5	0
1	0	6	5	1	1	8
2	3	1	8	5	8	1
0	9	1	2	3	1	4

if the above method is done to every column with stride of two
the answer will be as below

The output will be 3*3 . Max pooling takes maximum of the matrix to be the output. Hence the output will be
4 6 5
6 6 8
9 8 8

Which of following activation function can't be used at the output layer to classify an image

Sigmoid

Tanh

ReLU

ReLU cannot classify images because it will return the max(0 , z)
where as sigmoid gives 0 or 1 as output based on some threshold even the context is same with tanh we can classify by using -1 as negative classification and 1 as positive and lastly the 4th option is also doing the same.

1 if x > 5 else 0

None of the above

What's the role of pooling layer in CNN network

Pooling layers are used to reduce the dimensions of the feature maps it also reduces the number of parameters to learn and the amount of computation performed in a network.

Which of the following neural network training challenge can be solved using batch normalization?

Training is too slow

Restrict activations to become too high or low

Overfitting

It can solve all the above mentioned challenges

Both B and C

All of the above

Which of the following gives non-linearity to a neural network?

Stochastic Gradient Descent

As ReLu is a non-linear activation function which has a range from [max, 0] it provides non-linearity to the neural network.

Rectified Linear Unit

Convolution function

None of the above

Back Propagation cannot be applied when using pooling layers

Back Propagation can be applied to pooling layers also

True

False

You are building a CNN network and you realize that the pooling layer is causing significant loss of information. What would you do? Do you find proven alternatives of pooling layer?

increasing the number of channels in pooling operation.
Feature Scaling

Passed. Let's continue.